

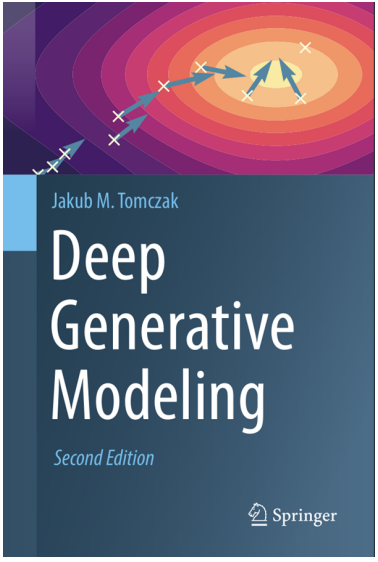
MACQUARIE University
SYDNEY·AUSTRALIA

Advanced Machine Learning - COMP8221

Week 2: Deep Generative Models - Part II

Dr. Qiongkai Xu, qiongkai.xu@mq.edu.au, School of Computing

Recommended textbook

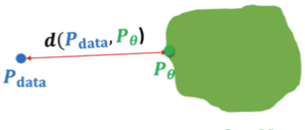


Learning a generative model

- We are give a training set of examples, e.g., images of dogs



$\mathbf{x}_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



- We want to learn a probability distribution $p(\mathbf{x})$ over images $\mathbf{x}$ such that:
  - **Generation**: if we sample $\mathbf{x} \sim p(\mathbf{x})$, $\mathbf{x}_{\text{new}}$ should look like a dog (sampling)
  - **Density estimation**: $p(\mathbf{x})$ should be high if $\mathbf{x}$ looks like a dog, and low otherwise
  - **Unsupervised representation learning**: We should be able to learn what these images have in common, i.e., ears, tails, etc (features)
- First question: how to represent $p(\mathbf{x})$. Second question: how to learn it.

Neural models for classification

- Setting: binary classification of $Y \in \{0, 1\}$ given input features $X \in \{0, 1\}^n$
- For classification, we care about $p(Y|X)$, and assume that

$$p(Y = 1|\mathbf{x}; \boldsymbol{\alpha}) = f(\mathbf{x}; \boldsymbol{\alpha}) \tag{1}$$

- **Logistic regression:** Let $z(\boldsymbol{\alpha}, \mathbf{x}) = \alpha_0 + \sum_{i=1}^{n} \alpha_i x_i$

$$p_{\text{logit}}(Y = 1|\mathbf{x}; \boldsymbol{\alpha}) = \sigma(z(\boldsymbol{\alpha}, \mathbf{x})), \text{ where } \sigma(z) = 1/(1 + e^{-z}) \tag{2}$$

- **Non-linear** dependence: let $\mathbf{h}(A, \mathbf{b}, \mathbf{x})$ be a non-linear transformation of the input features. $p_{\text{Neural}}(Y = 1|\mathbf{x}; \boldsymbol{\alpha}, A, \mathbf{b}) = \sigma(\alpha_0 + \sum_{i=1}^{n} \alpha_i \mathbf{h}_i)$. 1, more flexible; 2, more parameters: $A, \mathbf{b}, \boldsymbol{\alpha}$; 3, repeat multiple times to get a multilayer perceptron (neural network)

Motivating Example: MNIST

MACQUARIE University

- **Given**: a dataset $\mathcal{D}$ of handwritten digits (biniarized MNIST)



- Each image has $n = 28 \times 28 = 784$ pixels. Each pixel can either be black (0) or white (1).
- **Goal**: learn a probability distribution $p(\mathbf{x}) = p(x_1, \ldots, x_{784})$ over $\mathbf{x} \in \{0, 1\}^{784}$ such that when $\mathbf{x} \sim p(\mathbf{x})$, $\mathbf{x}$ looks like a digit
- Two step process
  - Parameterize a model family $\{p_\theta(x), \theta \in \Theta\}$
  - Search for model parameters θ based on training data $\mathcal{D}$

Autoregressive models

- We can pick an ordering of all the random variables, i.e., raster scan ordering of pixels from top-left x_1 to bottom-right $x_{n=784}$
- Without loss of generality, we can use chain rule for factorization

$$p(x_1, \ldots, x_{784}) = p(x_1)p(x_2|x_1)\cdots p(x_n|x_1, \cdots, x_{n-1}) \quad (3)$$

- Conditionals are too complex to be stored in a tabular form, we **assume**

$$p(x_1, \ldots, x_{784}) = p_{\text{CPT}}(x_1; \alpha^1)p_{\text{logit}}(x_2|x_1, \boldsymbol{\alpha}^2)p_{\text{logit}}(x_3|x_1, x_2; \boldsymbol{\alpha}^3)\cdots p_{\text{logit}}(x_n|x_1, \ldots, x_{n-1}; \boldsymbol{\alpha}^n) \quad (4)$$

- **Model assumption**: We are using parameterized functions (e.g., logistic regression above) to predict next pixel given all the previous ones. Called **autogressive model**

$$\text{E.g., } p_{\text{CPT}}(x_1 = 1; \alpha^1) = \alpha^1, p_{\text{logit}}(x_3 = 1|-) = \sigma(\alpha_0^3 + \alpha_1^3 x_1 + \alpha_2^3 x_2) \quad (5)$$

Fully Visiable Sigmoid Belief Network (FVSBN)

MACQUARIE University

- The conditional variables $x_i|x_1, \ldots, x_{i-1}$ are Bernoulli:

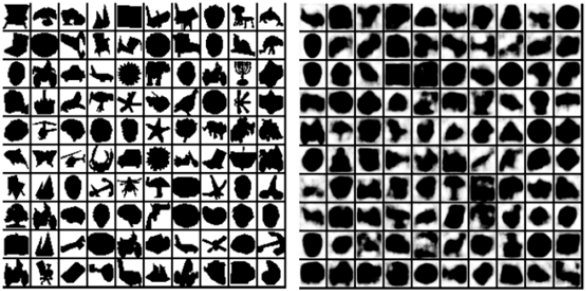
$$\widehat{x}_i = p(x_i = 1|x_1, \ldots, x_{i-1}; \boldsymbol{\alpha}^i) = \sigma(\alpha_0^i + \sum_{j=1}^{i-1} \alpha_j^i x_j) \tag{6}$$

- How to **evaluate** $p(x_1, \ldots, x_{784})$? Multiply all the conditionals (factors)

$$p(x_1 = 0, x_2 = 1, x_3 = 1) = (1 - \widehat{x}_1) \times \widehat{x}_2(x_1 = 0) \times \widehat{x}_3(x_1 = 0, x_2 = 1)$$

- How to sample from $p(x_1, \ldots, x_{784})$?
  1. Sample $\bar{x}_1 \sim p(x_1)$ ($np.random.choice([1, 0], p = [\widehat{x}_1, 1 - \widehat{x}_1])$)
  2. Sample $\bar{x}_2 \sim p(x_2|x_1 = \bar{x}_1)$
  3. Sample $\bar{x}_3 \sim p(x_3|x_1 = \bar{x}_1, x_2 = \bar{x}_2)$
- How many parameters? $1 + 2 + 3 + \ldots + n \approx n^2/2$.

FVSBN results



- Training data on the left (Caltech 101 Silhouettes). Samples from the model on the right.
- Figure from *Learning Deep Sigmoid Belief Networks with Data Augmentation*, 2015.

- To improve model: Use one-layer neural network instead of logistic regression:

$$\mathbf{h}_i = \sigma(A_i \mathbf{x}_{<i} + \mathbf{c}_i)$$

$$\hat{x}_i = p(x_i \mid x_1, \ldots, x_{i-1}; A_i, \mathbf{c}_i, \boldsymbol{\alpha}_i, b_i) = \sigma(\boldsymbol{\alpha}_i \mathbf{h}_i + b_i)$$

- For example:

$$\mathbf{h}_2 = \sigma\left(\begin{pmatrix} a_{11} \\ a_{12} \end{pmatrix} x_1 + \begin{pmatrix} c_1 \\ c_2 \end{pmatrix}\right), \quad \mathbf{h}_3 = \sigma\left(\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} c_1 \\ c_2 \end{pmatrix}\right)$$

- Tie weights to reduce the number of parameters and speed up computation:

$$\mathbf{h}_i = \sigma(W_{<i}\mathbf{x}_{<i} + \mathbf{c})$$

$$\hat{x}_i = p(x_i \mid x_1, \ldots, x_{i-1}) = \sigma(\alpha_i \mathbf{h}_i + b_i)$$

$$\mathbf{h}_2 = \sigma\left(\underbrace{\begin{pmatrix} \vdots \\ w_1 \\ \vdots \end{pmatrix}}_{W_{\cdot,<2}} x_1 + \mathbf{c}\right) \quad \mathbf{h}_3 = \sigma\left(\underbrace{\begin{pmatrix} \vdots & \vdots \\ w_1 & w_2 \\ \vdots & \vdots \end{pmatrix}}_{W_{\cdot,<3}} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) \quad \mathbf{h}_4 = \sigma\left(\underbrace{\begin{pmatrix} \vdots & \vdots & \vdots \\ w_1 & w_2 & w_3 \\ \vdots & \vdots & \vdots \end{pmatrix}}_{W_{\cdot,<4}} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}\right)$$

NADE Results



- Samples from a model trained on MNIST on the left. Conditional probabilities $\hat{x}_i$ on the right.

General Discrete Distributions

MACQUARIE University

- How to model non-binary discrete random variables $X_i \in \{1, \ldots, K\}$? E.g., pixel intensities from 0 to 255
- One solution: Let $\hat{x}_i$ parameterize a categorical distribution:

$$\mathbf{h}_i = \sigma(W_{<i}\mathbf{x}_{<i} + \mathbf{c})$$

$$p(x_i \mid x_1, \ldots, x_{i-1}) = \text{Cat}(p_i^1, \ldots, p_i^K)$$

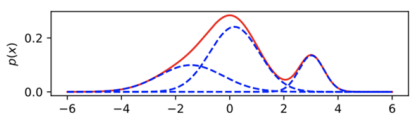
$$\tilde{\mathbf{x}}_i = (p_i^1, \ldots, p_i^K) = \text{softmax}(A_i\mathbf{h}_i + \mathbf{b}_i)$$

- Softmax generalizes the sigmoid function:

$$\text{softmax}(\mathbf{a}) = \left(\frac{\exp(a^1)}{\sum_i \exp(a^i)}, \ldots, \frac{\exp(a^K)}{\sum_i \exp(a^i)} \right)$$

- In NumPy: `np.exp(a) / np.sum(np.exp(a))`

- How to model continuous random variables $X_i \in \mathbb{R}$? E.g., speech signals
- Solution: Let $\hat{x}_i$ parameterize a continuous distribution, e.g., uniform mixture of K Gaussians



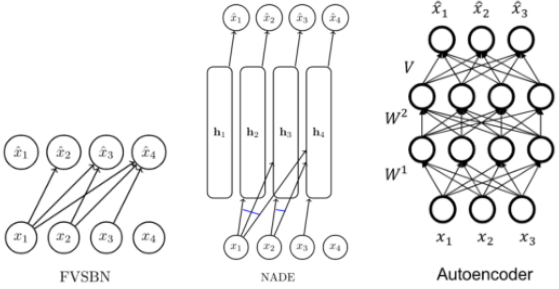
- How to model continuous random variables $X_i \in \mathbb{R}$? E.g., speech signals
- Solution: Let $\hat{x}_i$ parameterize a continuous distribution, e.g., mixture of K Gaussians:

$$
\begin{aligned}
p(x_i|x_1, \ldots, x_{i-1}) =& \frac{1}{K} \sum_{j=1}^{K} \mathcal{N}(x_i; \mu_i^j, \sigma_i^j) \\
\mathbf{h}_i =& \sigma(W_{\cdot,<i}\mathbf{x}_{\cdot,<i} + \mathbf{c}) \\
\widehat{x}_i =& (\mu_i^1, \ldots, \mu_i^K, \sigma_i^1, \ldots, \sigma_i^K) = f(\mathbf{h}_i)
\end{aligned}
$$

- $\hat{x}_i$ defines the mean and standard deviation of each of the K Gaussians (μ_i, σ_i), Can use exponential $\exp(\cdot)$ to ensure non-negativity

Autoregressive Models vs. Autoencoders

- FVSBN, NADE and autoencoder look similar:
  - An encoder $e(\cdot)$, e.g., $e(x) = \sigma(W^2(W^1 x + b^1) + b^2)$
  - A decoder such that $d(e(x)) = x$, e.g., $d(h) = \sigma(Vh + c)$



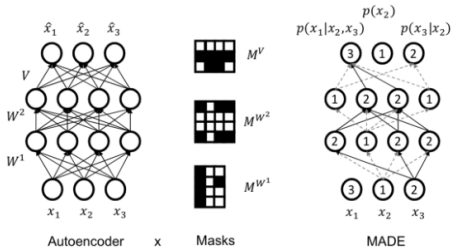
- Loss function for dataset $\mathcal{D}$
- e and d are constrained to avoid learning identity mappings. Hope that $e(x)$ is a meaningful, compressed representation of x (feature learning)
- A vanilla autoencoder is not a generative model: it does not define a distribution over x we can sample from to generate new data points.

Autoregressive Autoencoders

- On the surface, FVSBN and NADE look similar to an autoencoder. Can we get a generative model from an autoencoder?
- Need to ensure it corresponds to a valid Bayesian Network (DAG structure), i.e., need an ordering for chain rule. If ordering is $1, 2, 3$, then:
  - $\hat{x}_1$ cannot depend on any input $x = \{x_1, x_2, x_3\}$. Then at generation time, no input is needed to start
  - $\hat{x}_2$ can only depend on x_1 ...
- Bonus: Use a single neural network (with n inputs and outputs) to produce all parameters $\hat{x}$ in a single pass. In contrast, NADE requires n passes. Much more efficient on modern hardware.

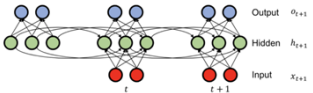
MADE: Masked Autoencoder for Distribution Estimation

- **Challenge**: An autoencoder that is autoregressive (DAG structure)
- **Solution**: Use masks to disallow certain paths (Germain et al., 2015). Suppose ordering is x_2, x_3, x_1, so $p(x_1, x_2, x_3) = p(x_2)p(x_3 \mid x_2)p(x_1 \mid x_2, x_3)$.



- Unit producing parameters for $x_2 = p(x_2)$ cannot depend on any input. Unit for $p(x_3 \mid x_2)$ only on x_2, etc.
- For each unit in a hidden layer, pick a random integer i in $[1, n-1]$. That unit depends only on the first i inputs (per chosen ordering).
- Add mask to preserve this invariant: connect to all units in the previous layer with a smaller or equal assigned number (strictly $<$ in final layer).

RNN: Recurrent Neural Nets



$$\begin{aligned} \text{Summary update rule: } h_{t+1} &= \tanh(W_{hh}h_t + W_{xh}x_{t+1}) \\ \text{Prediction: } o_{t+1} &= W_{hy}h_{t+1} \\ \text{Summary initalization: } h_0 &= \boldsymbol{b}_0 \end{aligned}$$

- **Challenge**: Model $p(x_t \mid x_{1:t-1}; \alpha^t)$. "History" $x_{1:t-1}$ keeps getting longer.
- **Idea**: Keep a summary and recursively update it:

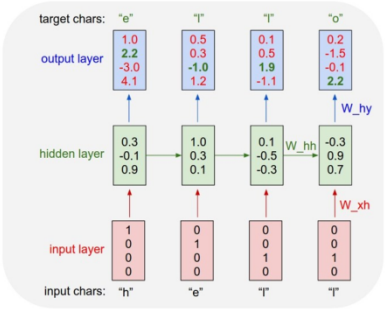
$$h_{t+1} = \tanh(W_h h_t + W_{xh}x_{t+1}), o_{t+1} = W_{hy}h_{t+1}, h_0 = b_0 \tag{7}$$

- Hidden layer h_t is a summary of inputs seen until time t.
- Output layer o_{t+1} specifies parameters for conditional $p(x_t \mid x_{1:t-1})$.
- Parameterized by b_0 (initialization), and matrices W_{hh}, W_{xh}, W_{hy}. Constant number of parameters w.r.t. n!

Example: Character RNN (from Andrei Karpathy)

MACQUARIE University

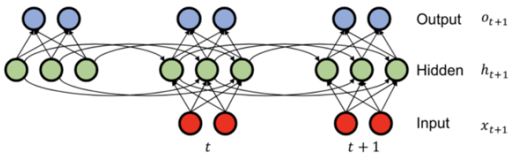
- Suppose $x_i \in \{h, e, l, o\}$. Use one-hot encoding:
  - h encoded as $[1, 0, 0, 0]$, e as $[0, 1, 0, 0]$, etc.
- Autoregressive:
  $p(x = \text{hello}) = p(x_1 = h)p(x_2 = e \mid x_1 = h)p(x_3 = l \mid x_1 = h, x_2 = e) \cdots p(x_5 = o \mid x_1 = h, x_2 = e, x_3 = l, x_4 = l)$
- For example:



$$p(x_2 = e \mid x_1 = h) = \text{softmax}(o_1) = \frac{\exp(2.2)}{\exp(1.0) + \cdots + \exp(4.1)}$$

$$o_1 = W_{hy} h_1, h_1 = \tanh(W_{hh} h_0 + W_{xh} x_1)$$

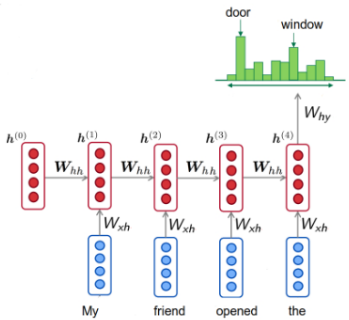
RNN: Recurrent Neural Nets



- **Pros**:
  - Can be applied to sequences of arbitrary length
  - Very general: For every computable function, there exists a finite RNN that can compute it
- **Cons**:
  - Still requires an ordering
  - Sequential likelihood evaluation (very slow for training)
  - Sequential generation (unavoidable in an autoregressive model)

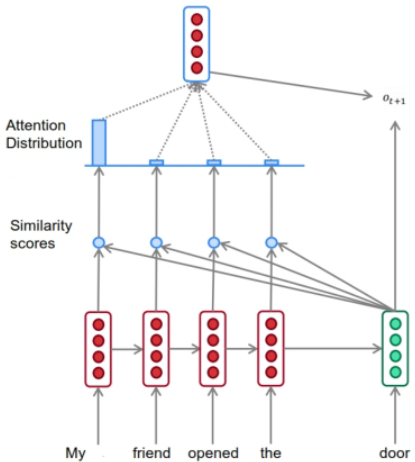
Issues with RNNs

- **Issues with RNN models**:
  - A single hidden vector needs to summarize all the (growing) history. For example, $h^{(4)}$ needs to summarize the meaning of "My friend opened the".
  - Sequential evaluation, cannot be parallelized
  - Exploding/vanishing gradients when accessing information from many steps back

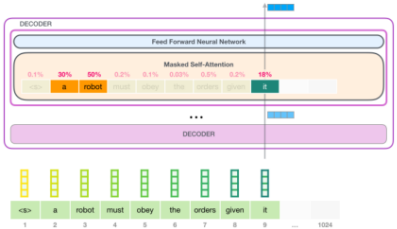


Attention Based Models

- Attention mechanism to compare a query vector to a set of key vectors:
  - Compare current hidden state (query) to all past hidden states (keys), e.g., by taking a dot product
  - Construct attention distribution to figure out what parts of the history are relevant, e.g., via a softmax
  - Construct a summary of the history, e.g., by weighted sum
  - Use summary and current hidden state to predict next token/word



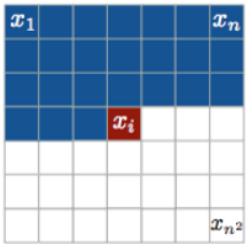
Generative Transformers



- Current state of the art (GPTs): Replace RNN with Transformer
  - Attention mechanisms to adaptively focus only on relevant context
  - Avoid recursive computation. Use only self-attention to enable parallelization
  - Needs masked self-attention to preserve autoregressive structure
- Demo: `https://transformer.huggingface.co/doc/gpt2-large`
- Demo: `https://huggingface.co/spaces/huggingface-projects/llama-2-13b-chat`

Pixel RNN (Oord et al., 2016)

- Model images pixel by pixel using raster scan order
- Each pixel conditional $p(x_t \mid x_{1:t-1})$ needs to specify 3 colors, and each conditional is a categorical random variable with 256 possible values
- Conditionals modeled using RNN variants: LSTMs + masking (like MADE)

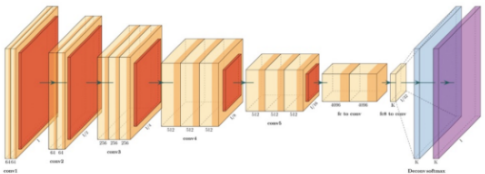


Pixel RNN



- Results on downsampled ImageNet. Very slow: sequential likelihood evaluation.
- Occluded, completions, original

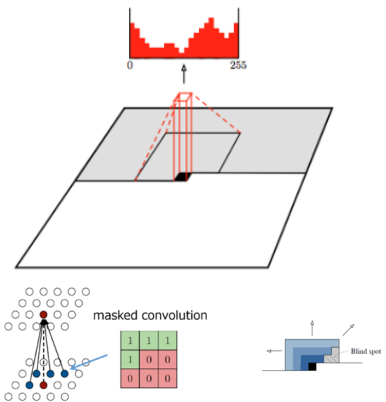
Convolutional Architectures

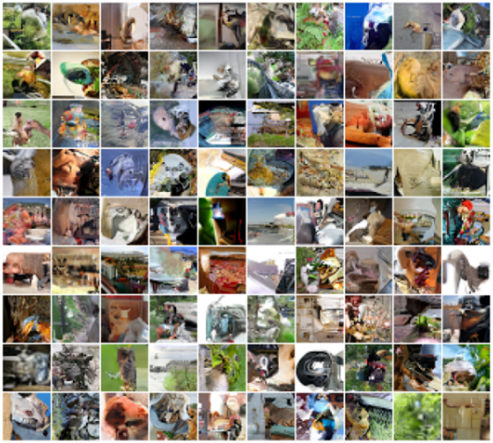


- Convolutions are natural for image data and easy to parallelize on modern hardware.

PixelCNN (Oord et al., 2016)

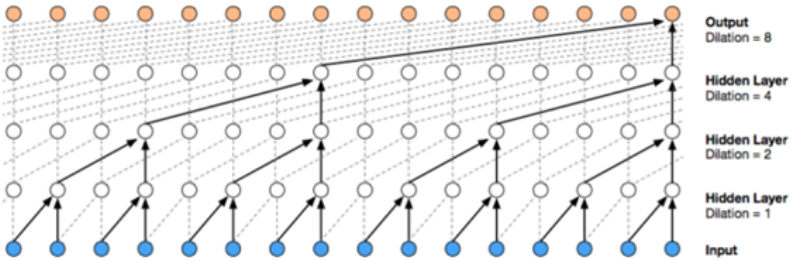
- **Idea**: Use convolutional architecture to predict the next pixel given context (a neighborhood of pixels).
- **Challenge**: Has to be autoregressive. Masked convolutions preserve raster scan order. Additional masking for color order.





- Sample from the model trained on ImageNet (32×32 pixels). Similar performance to PixelRNN, but much faster.

WaveNet (Oord et al., 2016)



- Very effective model for speech:
- Dilated convolutions increase the receptive field: kernel only touches the signal at every 2^d entries.

Summary of Autoregressive Models

MACQUARIE University

- **Easy to sample from**:
  - Sample $\bar{x}_0 \sim p(x_0)$
  - Sample $\bar{x}_1 \sim p(x_1 \mid x_0 = \bar{x}_0)$
  - ...
- **Easy to compute probability** $p(x = \bar{x})$:
  - Compute $p(x_0 = \bar{x}_0)$
  - Compute $p(x_1 = \bar{x}_1 \mid x_0 = \bar{x}_0)$
  - Multiply together (sum their logarithms)
  - ...
- Ideally, can compute all these terms in parallel for fast training
- Easy to extend to continuous variables, e.g., Gaussian conditionals $p(x_t \mid x_{<t}) = \mathcal{N}(\mu_\theta(x_{<t}), \Sigma_\theta(x_{<t}))$ or mixture of logistics
- No natural way to get features, cluster points, or do unsupervised learning

Recall: learning settings

MACQUARIE University

- Assumption: the domain is governed by some underlying distribution p_{data}
- We are given a dataset $\mathcal{D}$ of m samples from p_{data}
- Each sample is an assignment of values to (a subset of) the variables, e.g., $(X_{\text{bank}} = 1, X_{\text{dollar}} = 0, \ldots, Y = 1)$ or pixel intensities
- The standard assumption is that the data instances are independent and identically distributed (i.i.d.)
- We are also given a family of models $\mathcal{M}$, and our task is to learn some "good" distribution in this set:
  - For example, $\mathcal{M}$ could be all Bayes nets with a given graph structure, for all possible choices of the CPD tables
  - For example, a FVSBN for all possible choices of the logistic regression parameters, θ = concatenation of all logistic regression coefficients

Goal of Learning

MACQUARIE University

- The goal of learning is to return a model p_θ that precisely captures the distribution p_{data} from which our data was sampled
- This is in general not achievable because of:
  - Limited data only provides a rough approximation of the true underlying distribution
  - Computational reasons
- Example: Suppose we represent each image with a vector X of 784 binary variables (black vs. white pixel). How many possible states ($=$ possible images) in the model? $2^{784} \approx 10^{235}$. Even 10^7 training examples provide extremely sparse coverage!
- We want to select p_θ to construct the "best" approximation to the underlying distribution p_{data}
- What is "best"?

What is Best?

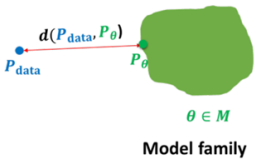
MACQUARIE University

- This depends on what we want to do:
- **Density estimation**: We are interested in the full distribution (so later we can compute whatever conditional probabilities we want)
- **Specific prediction tasks**: We are using the distribution to make a prediction
  - Is this email spam or not?
  - Structured prediction: Predict next frame in a video, or caption given an image
- **Structure or knowledge discovery**: We are interested in the model itself
  - How do some genes interact with each other?
  - What causes cancer?

Learning as Density Estimation

- We want to learn the full distribution so that later we can answer any probabilistic inference query
- In this setting, we can view the learning problem as density estimation
- We want to construct p_θ as "close" as possible to p_{data} (recall we assume we are given a dataset $\mathcal{D}$ of samples from p_{data})





- How do we evaluate "closeness"?

- How should we measure distance between distributions?
- The Kullback-Leibler divergence (KL-divergence) between two distributions p and q is defined as:

$$D(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

- $D(p\|q) \geq 0$ for all p, q, with equality if and only if $p(\cdot) = q(\cdot)$.
- The most commonly used divergence in deep generative models.

Main Idea in Monte Carlo Estimation

MACQUARIE University

- Express the quantity of interest as the expected value of a random variable:

$$E_{x\sim P}[g(x)] = \sum_x g(x)P(x)$$

- Generate T samples $\mathbf{x}^1, \ldots, \mathbf{x}^T$ from the distribution P with respect to which the expectation was taken
- Estimate the expected value from the samples using:

$$\hat{g}(\mathbf{x}^1, \ldots, \mathbf{x}^T) \triangleq \frac{1}{T}\sum_{t=1}^{T} g(\mathbf{x}^t)$$

  where $\mathbf{x}^1, \ldots, \mathbf{x}^T$ are independent samples from P
- Note: $\hat{g}$ is a random variable. Why?

Extending the MLE Principle to Autoregressive Models

- Given an autoregressive model with n variables and factorization:

$$p_\theta(x) = \prod_{i=1}^{n} p_{\text{neural}}(x_i \mid x_{<i}; \theta_i)$$

- $\theta = \{\theta_1, \cdots, \theta_n\}$ are the parameters of all the conditionals
- Training data $\mathcal{D} = \{\mathbf{x}^{(1)}, \cdots, \mathbf{x}^{(m)}\}$
- Maximum likelihood estimate of the parameters θ?
- Decomposition of Likelihood function:

$$\mathcal{L}(\theta, \mathcal{D}) = \prod_{j=1}^{m} p_\theta(\mathbf{x}^{(j)}) = \prod_{j=1}^{m} \prod_{i=1}^{n} p_{\text{neural}}(x_i^{(j)} \mid \mathbf{x}_{<i}^{(j)}, \theta_i)$$

- Goal: Maximize $\arg\max_\theta \mathcal{L}(\theta, \mathcal{D}) = \arg\max_\theta \log \mathcal{L}(\theta, \mathcal{D})$
- We no longer have a closed-form solution

MLE Learning: Gradient Descent

MACQUARIE University

- Log-likelihood:

$$\ell(\theta) = \log \mathcal{L}(\theta, \mathcal{D}) = \sum_{j=1}^{m} \sum_{i=1}^{n} \log p_{\text{neural}}(x_i^{(j)} \mid x_{<i}^{(j)}, \theta_i)$$

- Algorithm:
  1. Initialize θ^0 at random
  2. Compute $\nabla_\theta \ell(\theta)$ (by backpropagation)
  3. Update: $\theta^{t+1} = \theta^t + \alpha_t \nabla_\theta \ell(\theta)$

- Suppose we want to generate a set of variables $\mathbf{Y}$ given some others $\mathbf{X}$, e.g., text to speech
- We concentrate on modeling $p(\mathbf{Y} \mid \mathbf{X})$, and use a conditional loss function:

$$-\log p_\theta(\mathbf{y} \mid \mathbf{x})$$

- Since the loss function only depends on $p_\theta(\mathbf{y} \mid \mathbf{x})$, it suffices to estimate the conditional distribution, not the joint
- Example: Brown horse in grass field out in

Recap

- For autoregressive models, it is easy to compute $p_\theta(x)$
- Ideally, evaluate in parallel each conditional $\log p_{\text{neural}}(x_i^{(j)} \mid x_{<i}^{(j)}, \theta_i)$. Not like RNNs
- Natural to train them via maximum likelihood
- Higher log-likelihood doesn't necessarily mean better-looking samples
- Other ways of measuring similarity are possible (Generative Adversarial Networks, GANs)

Thank you!